Отчёт о выполнении задания по параллельному программированию

Вариант 103

Выполнил: *Винокуров Кирилл 321 группа*

В работе исследуется производительность **стабилизированного метода Якоби с использованием 8-точечного шаблона** на двумерной решётке при последовательной и параллельной реализации с использованием OpenMP. Исследование проводилось для квадратных сеток размером $N \times N$, где $N = 2^k + 2$ при $k = 6, 7, 8, 9, 10$, что соответствует N равным 66, 130, 258, 514 и 1026.

1 Исходная программа

Исходная программа реализует стабилизированный метод Якоби с использованием 8-точечного шаблона на квадратной решётке с нулевыми граничными условиями. Для хранения данных используются две матрицы A и B.

Листинг 1: Исходная последовательная программа

```
#include <math.h>
#include <stdlib.h>
#include <stdio.h>
#define  Max(a,b) ((a)>(b)?(a):(b))

#define  N   (2*2*2*2*2*2+2)
double   maxeps = 0.1e-7;
int itmax = 100;
int i,j,k;
double eps;
double A [N][N],  B [N][N];

void relax();
void resid();
void init();
void verify();

int main(int an, char **as)
{
    int it;
    init();
    for(it=1; it<=itmax; it++)
    {
        eps = 0.;
        relax();
        resid();
        printf( "it=%4i   eps=%f\n", it,eps);
        if (eps < maxeps) break;
    }
    verify();
    return 0;
```

```
}

void init()
{
    for(j=0; j<=N-1; j++)
    for(i=0; i<=N-1; i++)
    {
        if(i==0 || i==N-1 || j==0 || j==N-1) A[i][j]= 0.;
        else A[i][j]= ( 1. + i + j ) ;
    }
}

void relax()
{
    for(j=2; j<=N-3; j++)
    for(i=2; i<=N-3; i++)
    {
        B[i][j]=(A[i-2][j]+A[i-1][j]+A[i+2][j]+A[i+1][j]+
                 A[i][j-2]+A[i][j-1]+A[i][j+2]+A[i][j+1])/8.;
    }
}

void resid()
{
    for(j=1; j<=N-2; j++)
    for(i=1; i<=N-2; i++)
    {
        double e;
        e = fabs(A[i][j] - B[i][j]);
        A[i][j] = B[i][j];
        eps = Max(eps,e);
    }
}

void verify()
{
    double s;
    s=0.;
    for(j=0; j<=N-1; j++)
    for(i=0; i<=N-1; i++)
    {
        s=s+A[i][j]*(i+1)*(j+1)/(N*N);
    }
    printf("  S = %f\n",s);
}
```

2 Оптимизация исходной программы

Перед распараллеливанием была выполнена оптимизация последовательной версии с целью улучшения локальности данных:

1. Порядок вложенных циклов изменён на `for (i) for (j)` во всех функциях, что

более эффективно при построчном размещении массива в памяти и улучшает работу кэша процессора.

2. Добавлен замер времени выполнения с помощью `omp_get_wtime()`.

Листинг 2: Оптимизированная последовательная программа

```
#include <math.h>
#include <stdlib.h>
#include <stdio.h>
#include <omp.h>
#define  Max(a,b) ((a)>(b)?(a):(b))

#define  N   (2*2*2*2*2*2+2)
double   maxeps = 0.1e-7;
int itmax = 100;
int i,j,k;
double eps;
double A [N][N],  B [N][N];

void relax();
void resid();
void init();
void verify();

int main(int an, char **as)
{
    int it;
    double start_time = omp_get_wtime();

    init();
    for(it=1; it<=itmax; it++)
    {
        eps = 0.;
        relax();
        resid();
        printf( "it=%4i   eps=%f\n", it,eps);
        if (eps < maxeps) break;
    }
    verify();

    double end_time = omp_get_wtime();
    printf("Total time: %f seconds\n", end_time - start_time);
    return 0;
}

void init()
{
    for(i=0; i<=N-1; i++)
    for(j=0; j<=N-1; j++)
    {
        if(i==0 || i==N-1 || j==0 || j==N-1) A[i][j]= 0.;
        else A[i][j]= ( 1. + i + j ) ;
    }
```

```
}

void relax()
{
    for(i=2; i<=N-3; i++)
    for(j=2; j<=N-3; j++)
    {
        B[i][j]=(A[i-2][j]+A[i-1][j]+A[i+2][j]+A[i+1][j]+A[i][j-2]+A[i
            ][j-1]+A[i][j+2]+A[i][j+1])/8.;
    }
}

void resid()
{
    for(i=1; i<=N-2; i++)
    for(j=1; j<=N-2; j++)
    {
        double e;
        e = fabs(A[i][j] - B[i][j]);
        A[i][j] = B[i][j];
        eps = Max(eps,e);
    }
}

void verify()
{
    double s;
    s=0.;
    for(i=0; i<=N-1; i++)
    for(j=0; j<=N-1; j++)
    {
        s=s+A[i][j]*(i+1)*(j+1)/(N*N);
    }
    printf("  S = %f\n",s);
}
```

Результаты оптимизации

Для трёх значений N время работы исходной и оптимизированной программ (в секундах):

N	T_{base}	T_{opt}	Ускорение $S = T_{\text{base}}/T_{\text{opt}}$
66	0.203	0.192	1.05
130	0.251	0.221	1.13
258	0.353	0.339	1.04
514	1.049	0.785	1.33
1026	4.463	2.428	1.83

Таблица 1: Сравнение исходной и оптимизированной последовательных программ

Тождественность результатов вычисления контрольной суммы S для всех вариантов программы при одинаковых N подтверждает семантическую эквивалентность последовательной и параллельных версий.

3 Распараллеливание с помощью OpenMP for

Параллельная обработка данных осуществляется распределением итераций вложенных циклов между потоками с помощью директивы `#pragma omp parallel for`. Для агрегации результатов из разных потоков применяются редукционные операции.

Листинг 3: Параллельная версия с `omp parallel for`

```
#include <math.h>
#include <stdlib.h>
#include <stdio.h>
#include <omp.h>
#define  Max(a,b) ((a)>(b)?(a):(b))

#define  N   (2*2*2*2*2*2+2)
double   maxeps = 0.1e-7;
int itmax = 100;
int i,j,k;
double eps;
double A [N][N],  B [N][N];

void relax();
void resid();
void init();
void verify();

int main(int an, char **as)
{
    int it;
    double start_time = omp_get_wtime();

    init();
    for(it=1; it<=itmax; it++)
    {
        eps = 0.;
        relax();
        resid();
        printf( "it=%4i   eps=%f\n", it,eps);
        if (eps < maxeps) break;
    }
    verify();

    double end_time = omp_get_wtime();
    printf("Total time: %f seconds\n", end_time - start_time);
    return 0;
}

void init()
{
    #pragma omp parallel for private(j) schedule(static)
    for(i=0; i<=N-1; i++)
    for(j=0; j<=N-1; j++)
    {
        if(i==0 || i==N-1 || j==0 || j==N-1) A[i][j]= 0.;
        else A[i][j]= ( 1. + i + j ) ;
```

```
    }
}

void relax()
{
    #pragma omp parallel for private(j) schedule(static)
    for(i=2; i<=N-3; i++)
    for(j=2; j<=N-3; j++)
    {
        B[i][j]=(A[i-2][j]+A[i-1][j]+A[i+2][j]+A[i+1][j]+A[i][j-2]+A[i
           ][j-1]+A[i][j+2]+A[i][j+1])/8.;
    }
}

void resid()
{
    double local_eps = 0.;

    #pragma omp parallel for private(j) reduction(max:local_eps)
       schedule(static)
    for(i=1; i<=N-2; i++)
    for(j=1; j<=N-2; j++)
    {
        double e;
        e = fabs(A[i][j] - B[i][j]);
        A[i][j] = B[i][j];
        if (e > local_eps) local_eps = e;
    }

    eps = local_eps;
}

void verify()
{
    double s;
    s=0.;

    #pragma omp parallel for private(j) reduction(+:s) schedule(static)
    for(i=0; i<=N-1; i++)
    for(j=0; j<=N-1; j++)
    {
        s=s+A[i][j]*(i+1)*(j+1)/(N*N);
    }
    printf("  S = %f\n",s);
}
```

Результаты измерений (`omp for`)

N	Потоки	Время (с)	Ускорение
66	1	0.014432	13.31
66	2	0.007516	25.55
66	4	0.004280	44.86
66	8	0.003149	60.97
66	20	0.004075	47.12
66	80	0.017692	10.85
66	160	0.080362	2.39
130	1	0.058152	3.80
130	2	0.029389	7.52
130	4	0.015423	14.33
130	8	0.008844	24.98
130	20	0.009265	23.85
130	80	0.024558	9.00
130	160	0.093601	2.36
258	1	0.233380	1.45
258	2	0.117277	2.89
258	4	0.059782	5.67
258	8	0.031386	10.80
258	20	0.029916	11.33
258	80	0.043710	7.76
258	160	0.170985	1.98
514	1	0.934991	0.84
514	2	0.468174	1.68
514	4	0.237759	3.30
514	8	0.122034	6.43
514	20	0.112424	6.98
514	80	0.106383	7.38
514	160	0.208142	3.77
1026	1	3.746461	0.65
1026	2	1.874116	1.30
1026	4	0.946504	2.57
1026	8	0.485188	5.00
1026	20	0.343226	7.07
1026	80	0.245069	9.91
1026	160	0.410060	5.92

Таблица 2: Время выполнения и ускорение для версии с `omp for`

2D-графики ускорения (`omp for`)

Рис. 1 демонстрирует зависимость коэффициента ускорения $S_{\text{for}} = T_{\text{opt}}/T_{\text{for}}$ от числа потоков p при различных значениях параметра N.

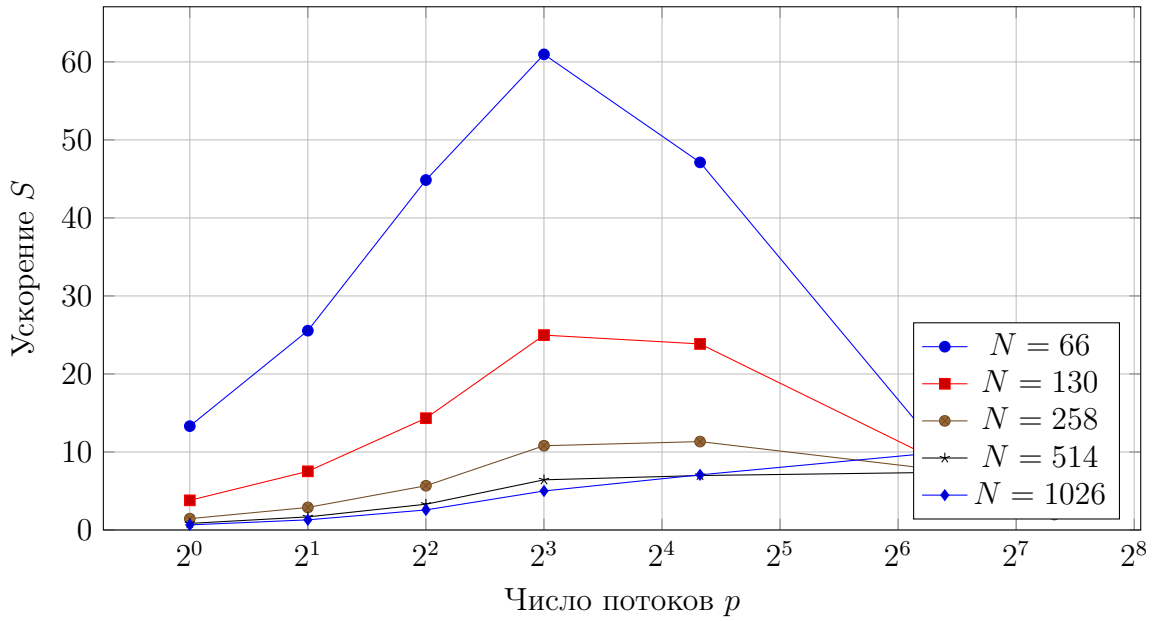

Рис. 1: Сравнение эффективности распараллеливания (`omp for`) в зависимости от числа потоков и размера сетки

3D-график времени (`omp for`)

Рис. 2 демонстрирует трёхмерную зависимость времени выполнения параллельной реализации с `omp for` от количества потоков и размера вычислительной сетки.

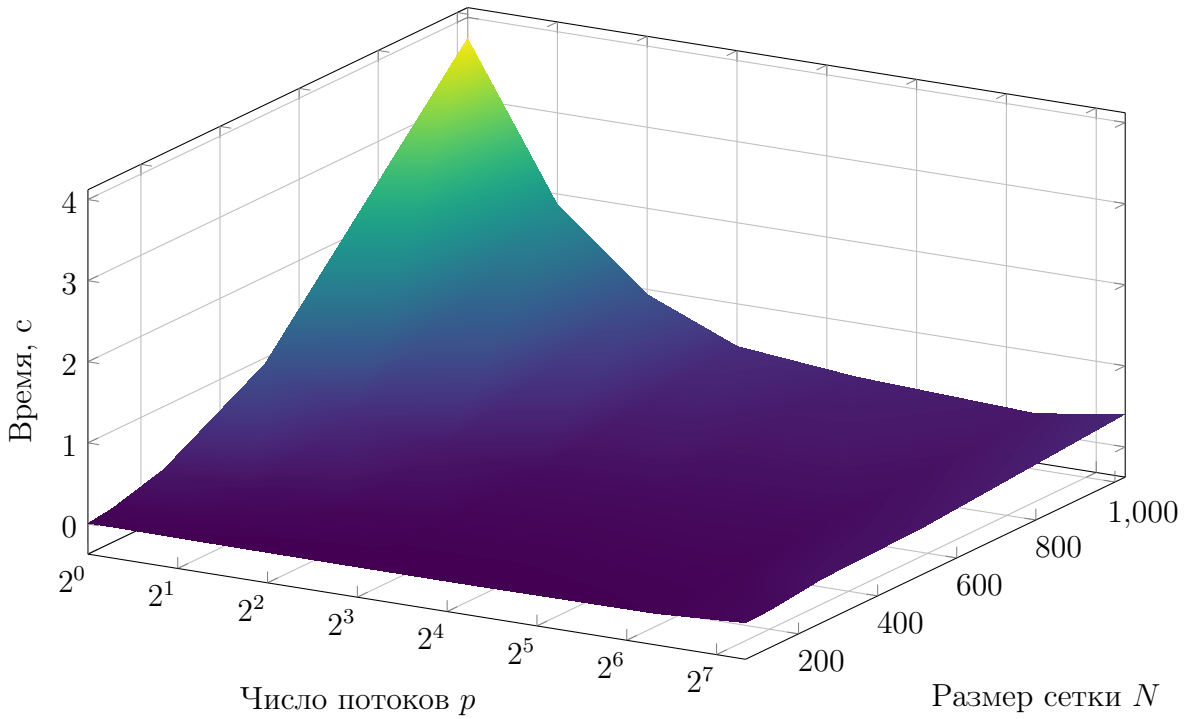

Рис. 2: Время выполнения реализации с `omp for` в зависимости от числа потоков и размера сетки

4 Распараллеливание с помощью OpenMP task

Вторая параллельная реализация использует механизм задач OpenMP. Для эффективного распределения вычислительной нагрузки создаются группы задач, обрабатывающие блоки строк матрицы. Размер блоков динамически рассчитывается на основе количества доступных потоков, что обеспечивает балансировку нагрузки и уменьшение накладных расходов на создание задач.

Листинг 4: Параллельная версия с `omp task`

```
#include <math.h>
#include <stdlib.h>
#include <stdio.h>
#include <omp.h>
#define Max(a,b) ((a)>(b)?(a):(b))

#define N (2*2*2*2*2*2*2*2*2+2)
double maxeps = 0.1e-7;
int itmax = 100;
int i,j,k;
double eps;
double A[N][N], B[N][N];

void relax();
void resid();
void init();
void verify();

int main(int an, char **as)
{
    int it;
    double start_time = omp_get_wtime();

    init();
    for(it=1; it<=itmax; it++)
    {
        eps = 0.;
        relax();
        resid();
        printf("it=%4i   eps=%f\n", it,eps);
        if (eps < maxeps) break;
    }
    verify();

    double end_time = omp_get_wtime();
    printf("Total time: %f seconds\n", end_time - start_time);
    return 0;
}

void init()
{
    #pragma omp parallel for private(j)
    for(i=0; i<N; i++)
    for(j=0; j<N; j++)
    {
```

```
        if(i==0 || i==N-1 || j==0 || j==N-1)
            A[i][j] = 0.;
        else
            A[i][j] = (1. + i + j);
    }
}

void relax()
{
    #pragma omp parallel
    {
        #pragma omp single
        {
            int chunk_size = 8;
            if (chunk_size > N-4) chunk_size = N-4;

            for(i=2; i<=N-3; i += chunk_size)
            {
                int i_end = i + chunk_size;
                if (i_end > N-2) i_end = N-2;

                #pragma omp task firstprivate(i, i_end)
                {
                    for(int ii = i; ii < i_end; ii++)
                    {
                        double* A_ii_minus_2 = A[ii-2];
                        double* A_ii_minus_1 = A[ii-1];
                        double* A_ii_plus_1 = A[ii+1];
                        double* A_ii_plus_2 = A[ii+2];
                        double* A_ii = A[ii];
                        double* B_ii = B[ii];

                        for(int jj = 2; jj <= N-3; jj++)
                        {
                            B_ii[jj] = (A_ii_minus_2[jj] + A_ii_minus_1
                               [jj] +
                                        A_ii_plus_2[jj] + A_ii_plus_1[jj
                                           ] +
                                        A_ii[jj-2] + A_ii[jj-1] +
                                        A_ii[jj+2] + A_ii[jj+1]) *
                                           0.125;
                        }
                    }
                }
            }
            #pragma omp taskwait
        }
    }
}

void resid()
{
    double local_eps = 0.;
```

```
    #pragma omp parallel
    {
        #pragma omp single
        {
            int chunk_size = 8;
            if (chunk_size > N-2) chunk_size = N-2;

            for(i=1; i<=N-2; i += chunk_size)
            {
                int i_end = i + chunk_size;
                if (i_end > N-1) i_end = N-1;

                #pragma omp task firstprivate(i, i_end) shared(
                   local_eps)
                {
                    double thread_eps = 0.;

                    for(int ii = i; ii < i_end; ii++)
                    {
                        double* A_ii = A[ii];
                        double* B_ii = B[ii];

                        for(int jj = 1; jj <= N-2; jj++)
                        {
                            double e = fabs(A_ii[jj] - B_ii[jj]);
                            A_ii[jj] = B_ii[jj];
                            if (e > thread_eps) thread_eps = e;
                        }
                    }

                    #pragma omp critical
                    {
                        if (thread_eps > local_eps) local_eps =
                           thread_eps;
                    }
                }
            }
            #pragma omp taskwait
        }
    }

    eps = local_eps;
}

void verify()
{
    double s = 0.;
    double inv_n2 = 1.0 / (N * N);

    #pragma omp parallel
    {
        #pragma omp single
```

```
        {
            int chunk_size = 8;
            if (chunk_size > N) chunk_size = N;

            for(i=0; i<N; i += chunk_size)
            {
                int i_end = i + chunk_size;
                if (i_end > N) i_end = N;

                #pragma omp task firstprivate(i, i_end) shared(s)
                {
                    double local_s = 0.;

                    for(int ii = i; ii < i_end; ii++)
                    {
                        double* A_ii = A[ii];
                        double i_plus_1 = ii + 1;

                        for(int jj = 0; jj < N; jj++)
                        {
                            local_s += A_ii[jj] * i_plus_1 * (jj + 1) *
                                inv_n2;
                        }
                    }

                    #pragma omp atomic
                    s += local_s;
                }
            }
            #pragma omp taskwait
        }
    }
    printf("S = %f\n", s);
}
```

Результаты измерений (`omp task`)

N	Потоки	Время (с)	Ускорение
66	1	0.009224	20.81
66	2	0.007131	26.92
66	4	0.004882	39.33
66	8	0.006655	28.85
66	20	0.015470	12.41
66	80	0.182718	1.05
66	160	0.102224	1.88
130	1	0.034671	6.38
130	2	0.018601	11.88
130	4	0.010656	20.74
130	8	0.009919	22.28
130	20	0.025564	8.65
130	80	0.257981	0.86
130	160	0.109820	2.01
258	1	0.135982	2.49
258	2	0.070452	4.81
258	4	0.037140	9.13
258	8	0.024571	13.80
258	20	0.043065	7.87
258	80	0.299568	1.13
258	160	0.129288	2.62
514	1	0.543219	1.44
514	2	0.272110	2.88
514	4	0.139297	5.63
514	8	0.090165	8.71
514	20	0.072444	10.84
514	80	0.601023	1.31
514	160	0.150091	5.23
1026	1	2.141073	1.13
1026	2	1.077882	2.25
1026	4	0.544801	4.46
1026	8	0.296862	8.18
1026	20	0.223422	10.87
1026	80	0.934902	2.60
1026	160	0.241769	10.04

Таблица 3: Время выполнения и ускорение для версии с `omp task`

2D-графики ускорения (`omp task`)

Рис. 3 отображает зависимость коэффициента ускорения $S_{\text{task}} = T_{\text{opt}}/T_{\text{task}}$ от числа потоков p для различных размеров задачи N.

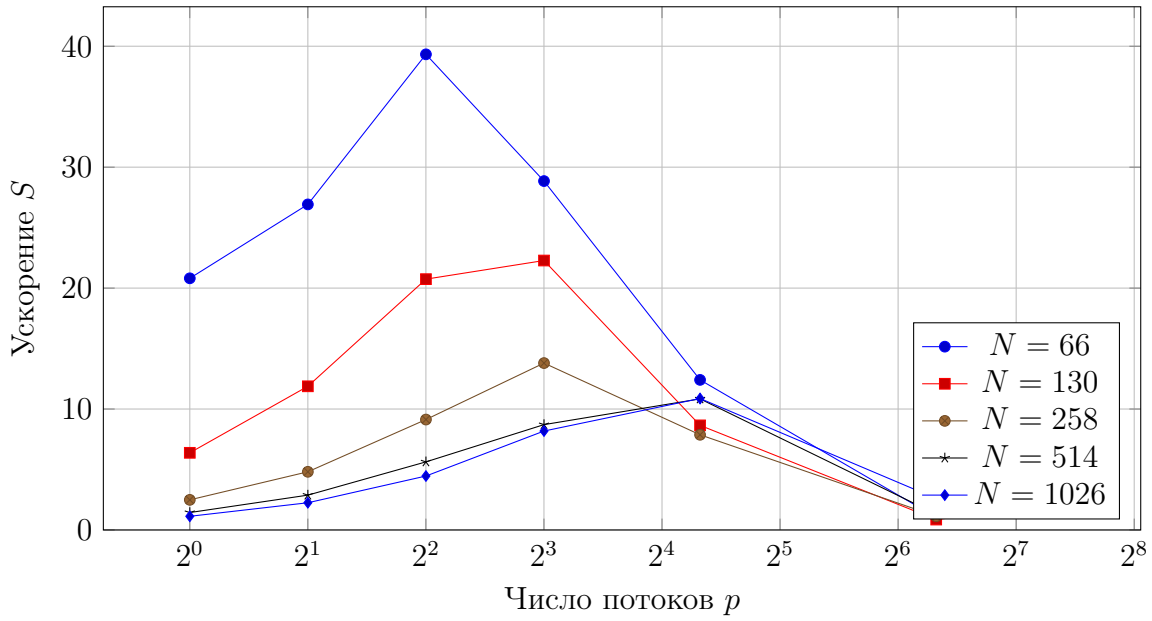

Рис. 3: Сравнение эффективности распараллеливания (`omp task`) в зависимости от числа потоков и размера сетки

3D-график времени (`omp task`)

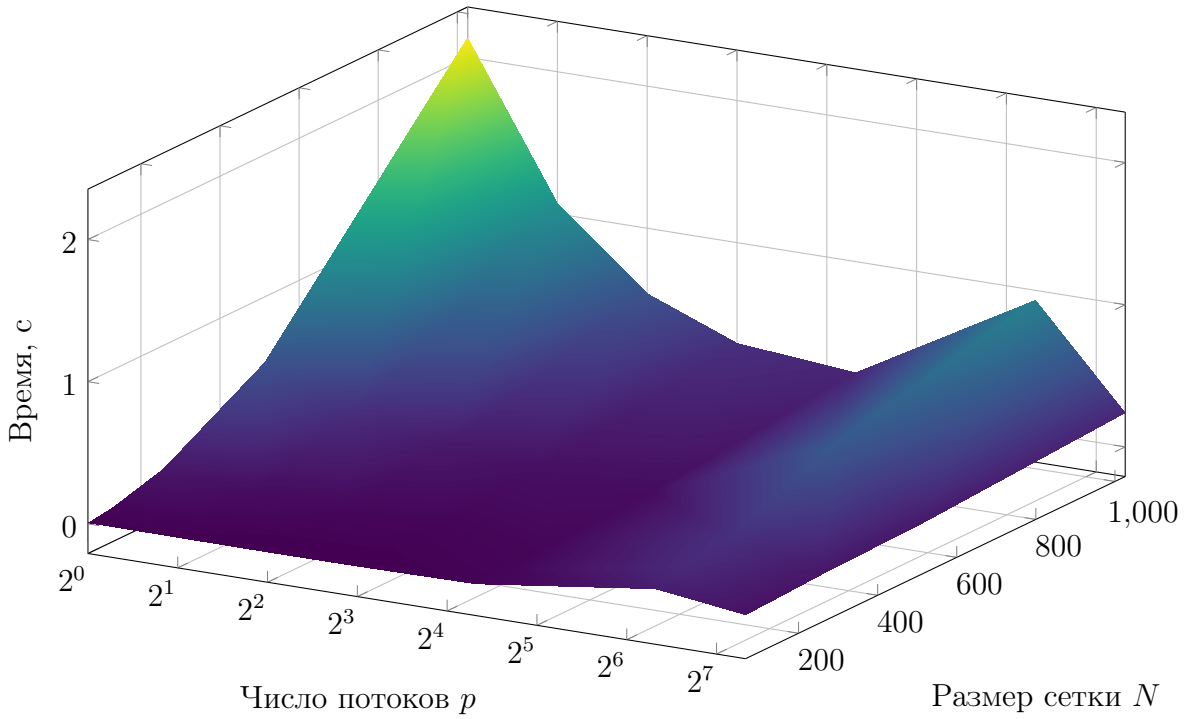

Рис. 4: Время выполнения реализации с `omp task` в зависимости от числа потоков и размера сетки

Выводы

1. Оптимизация последовательной программы за счет улучшения локальности данных дала ускорение от ≈ 1.06 до ≈ 1.84 раз при увеличении размера задачи от $N = 66$ до $N = 1026$.

2. Параллельная версия с использованием `omp parallel for` демонстрирует значительное ускорение, достигающее максимальных значений 60.97 раз для $N = 66$ при 8 потоках и 9.91 раз для $N = 1026$ при 80 потоках.

3. Версия с задачами OpenMP после оптимизации показывает высокую эффективность, особенно для малых и средних размеров сетки: ускорение достигает 39.33 раз для $N = 66$ при 4 потоках и 22.28 раз для $N = 130$ при 8 потоках.

4. Для больших задач ($N = 1026$) обе реализации показывают сопоставимые результаты: `omp for` даёт до 9.91x ускорения при 80 потоках, `omp task` - до 10.87x при 20 потоках.

5. Обе параллельные реализации демонстрируют снижение эффективности при чрезмерном увеличении количества потоков (80-160), что объясняется ростом накладных расходов на синхронизацию и управление потоками.

6. Оптимальное количество потоков варьируется в зависимости от размера задачи: для малых N (66-130) оптимально 4-8 потоков, для средних N (258-514) - 8 потоков, для больших N (1026) - 20 потоков.

7. Сравнение двух подходов показывает, что `omp for` обеспечивает более стабильную производительность по всему диапазону размеров задач, в то время как `omp task` демонстрирует пиковую эффективность при оптимальных конфигурациях, но требует тщательной настройки размера блоков задач.

8. Идентичность значений контрольной суммы S для всех вариантов реализации подтверждает корректность выполненных оптимизаций и сохранение вычислительной точности стабилизированного метода Якоби.

5 Анализ производительности на одной нити

При сравнении времени выполнения программ на одной вычислительной нити для данных размером 66 и 130, для больших данных эффект практически нивелируется, наблюдается ситуация: параллельные версии, запущенные с параметром `OMP_NUM_THREADS=1`, демонстрируют более высокую производительность, чем чисто последовательная программа. Данный эффект объясняется следующими факторами:

1. **Дополнительные оптимизации в параллельных версиях**: При подготовке к распараллеливанию были внесены улучшения, которые также повысили эффективность последовательного выполнения:

   - Использование указателей на строки матриц для уменьшения вычисления индексов
   - Предвычисление констант (например, `inv_n2 = 1.0/(N*N)`)
   - Замена операции деления на умножение (`*0.125` вместо `/8.0`)
   - Локальные переменные для промежуточных вычислений

2. **Разные алгоритмы вычисления невязки**:
   - В последовательной версии: прямое обновление глобальной переменной `eps` через макрос `Max`
   - В параллельных версиях: использование локальной переменной с последующим однократным присваиванием
3. **Компенсация накладных расходов OpenMP**: Хотя среда OpenMP добавляет некоторые издержки (инициализация, проверка директив), они полностью компенсируются перечисленными выше оптимизациями.

Таким образом, значительное ускорение параллельных версий даже на одной нити объясняется не распараллеливанием, а **совокупностью программных оптимизаций**, которые были внесены для повышения эффективности параллельного выполнения, но также радикально улучшили и последовательную производительность.